

Code Sense

Deployment & Security Dossier

Offline AI Code-Review & SBOM Scanner (Desktop)

Document version: v1.4

Date: 2026-05-28

Classification: Confidential

Prepared for: Bank Information Security / Vendor Risk / Endpoint Engineering

Deployment model: Single self-contained Windows desktop application, fully offline

*This document contains the product Bill of Materials, security posture, data-handling model, and risk register for review prior to endpoint deployment. Items marked **[VERIFY]** require vendor confirmation before sign-off.*

1. Executive summary

Code Sense is a security code-review tool that runs **entirely** on the user's laptop with no network connectivity. It scans source code (SAST, via a bundled local AI model) and software dependencies (SCA/SBOM, via bundled Syft/Grype) and stores all results in a local database. **No source code, scan results, or telemetry ever leaves the machine** — there is no cloud backend, no external AI/inference API, and no licensing "phone home."

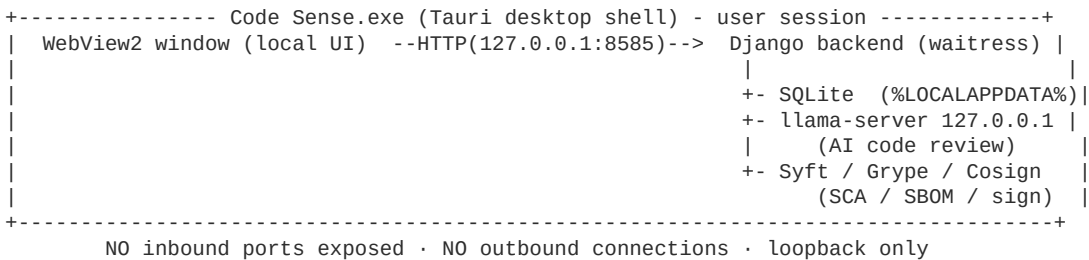
This is the primary control relevant to a bank: code under review (potentially proprietary or regulated) is processed **locally only**. The application binds exclusively to the loopback interface (127.0.0.1) and makes **zero outbound or inbound network connections** at runtime.

The package is delivered as a signed Windows installer that bundles all runtime components (application, AI model, scanner binaries, embedded database, web runtime). It is licensed for a fixed **90-day** term from first launch, after which it enters a read-only state.

2. Product overview

Attribute	Detail
Function	SAST (AI code review) + SCA/SBOM (dependency CVE + license scanning)
Form factor	Native Windows desktop app (Tauri shell hosting a local web UI)
Backend	Django (Python) served by waitress on 127.0.0.1:8585 (loopback only)
AI inference	Local llama.cpp server (127.0.0.1, loopback only) — no external API
Datastore	Embedded SQLite file in the per-user profile
External services	None at runtime (no cloud, no MongoDB, no central license server)
Multi-user	Single-user desktop; in-app RBAC (admin/manager/user)

3. Deployment architecture & data flow



Data flow (all local):

1. User selects source (ZIP upload or local path) or an SBOM file inside the app.
2. Backend runs AST metrics + the local AI model (SAST) and/or Syft->Grype (SCA) — all on-device.
3. Findings (CVEs, CWEs, licenses, severities) are written to the local SQLite database.
4. The UI reads them back over loopback. Reports can be exported to local files (Excel/CSV) by the user.

No data egress path exists. The application contains no code that opens outbound sockets to non-loopback hosts at runtime. (Network access occurs only on the **vendor's build machine** when assembling the installer — see §7.9 — never on the bank laptop.)

4. Network & connectivity requirements

Direction	Requirement
Outbound (internet)	None. Works fully air-gapped.
Outbound (LAN)	None.
Inbound	None. Listeners bind to 127.0.0.1 only (ALLOWED_HOSTS=127.0.0.1, localhost).
Loopback (same host)	127.0.0.1:8585 (backend), one private loopback port for the AI server.
Proxy/firewall changes	Not required. No allow-listing needed.

5. Data handling, privacy & residency

Topic	Detail
Data processed	Source code / repositories and SBOM files the user submits; scan findings.
Data residency	100% local to the laptop. Nothing is transmitted off-device.
Storage location	%LOCALAPPDATA%\com.codesense.desktop\ (SQLite DB, license stamp, signing keys, logs).
PII	Only the operator account (name, email, bcrypt-hashed password) entered during setup. No end-customer PII is collected by the tool.
Telemetry / analytics	None. No usage data, crash reporting, or beaconing.
Third-party data sharing	None.
Retention / deletion	Data persists in the local SQLite file until the user deletes records or uninstalls; uninstall + removal of the data directory fully purges it.
Encryption at rest	Relies on the endpoint's full-disk encryption (e.g., BitLocker) per bank policy. The app does not implement its own DB encryption. [VERIFY] confirm BitLocker is enforced on target laptops.

6. Authentication, authorization & application security

Control	Implementation
Authentication	JWT (HS256, 60-min expiry); credentials verified with bcrypt .
First-run setup	On first launch the user creates the admin account in-app. No default credentials are shipped. A strong-password policy is enforced (min 8 chars; upper/lower/digit/special).
Authorization	Role-based access control (admin / manager / user) with per-action permission flags enforced server-side via decorators.
Session secret	Django SECRET_KEY is generated uniquely per install on first run and persisted locally (no shared/hardcoded key).
Transport	Loopback only; traffic does not traverse any network.
Hardening	DEBUG=False in the shipped build; ALLOWED_HOSTS restricted to loopback; CORS restricted to the app's own origin.

Control	Implementation
Independent review	An internal security review of the codebase found no exploitable vulnerabilities (no SQL/command injection, no unsafe deserialization, parameterized ORM throughout). Defense-in-depth hardening was applied to reviewed items (segment-aware request gating, per-install license key derivation, atomic admin bootstrap). See §11.

7. Bill of Materials (BOM / SBOM)

A machine-readable **CycloneDX SBOM can be generated on demand** using the Syft binary bundled with the product (the same tool the app uses for scanning). The tables below enumerate the first-party components and all bundled third parties with versions, licenses, and sources.

7.1 First-party application components

Component	Description	Language
Code Sense backend	REST API, scan orchestration, RBAC, licensing	Python / Django
Code Sense frontend	Local web UI	TypeScript / React
Tauri desktop shell	Window, tray, sidecar process management	Rust
AI code-review model	Fine-tuned LLM ("astra-code-reviewer")	model weights (GGUF)

7.2 Backend runtime dependencies (Python 3.13)

All pinned to the exact versions resolved into the shipped build. All OSI-approved / permissive.

Package	Version	License
Django	5.2.14	BSD-3-Clause
djangorestframework	3.15.2	BSD-3-Clause
django-cors-headers	4.9.0	MIT
PyJWT	2.13.0	MIT
bcrypt	4.3.0	Apache-2.0
cryptography	44.0.3	Apache-2.0 / BSD-3-Clause
waitress	3.0.2	ZPL-2.1
requests	2.34.2	Apache-2.0
httpx	0.27.2	BSD-3-Clause
openpyxl	3.1.5	MIT
python-dotenv	1.2.2	BSD-3-Clause
asgiref	3.11.1	BSD-3-Clause
sqlparse	0.5.5	BSD-3-Clause
urllib3	2.7.0	MIT
certifi	2026.5.20	MPL-2.0

Package	Version	License
idna	3.16	BSD-3-Clause
charset-normalizer	3.4.7	MIT
anyio	4.13.0	MIT
sniffio	1.3.1	MIT / Apache-2.0
h11	0.16.0	MIT
httpcore	1.0.9	BSD-3-Clause
cffi	2.0.0	MIT
pycparser	3.0	BSD-3-Clause
dnspython	2.8.0	ISC
et-xmlfile	2.0.0	MIT

Note: pymongo/bson were **removed** during the migration to a self-contained SQLite store;

no MongoDB driver or server is present. (certifi/MPL and waitress/ZPL are used unmodified as

libraries; no copyleft obligation is triggered for this usage.)

7.3 Frontend dependencies (Node / bundled into static assets; key direct deps)

Package	Version	License
react / react-dom	19.1.0	MIT
@tanstack/react-router	1.123.2	MIT
@tanstack/react-query	5.83.0	MIT
@tanstack/react-form	1.14.1	MIT
axios	1.10.0	MIT
@radix-ui/react-* (dialog, select, dropdown, tooltip, switch, collapsible, slot)	1.x–2.x	MIT
tailwindcss	4.1.11	MIT
recharts	3.0.2	MIT
lucide-react	0.525.0	ISC
sonner	2.0.5	MIT
clsx / class-variance-authority / tailwind-merge / next-themes	current	MIT
vite (build)	7.0.0	MIT
typescript (build)	5.8.3	Apache-2.0

The full transitive dependency tree is pinned by client/package-lock.json / bun.lock and is

available as a CycloneDX SBOM on request.

7.4 Desktop shell (Rust / Tauri crates)

Crate	Version	License
tauri	2.x	MIT / Apache-2.0
tauri-plugin-shell	2.x	MIT / Apache-2.0

Crate	Version	License
tauri-build	2.x	MIT / Apache-2.0
serde / serde_json	1.x	MIT / Apache-2.0

Full crate lock is pinned by `client/src-tauri/Cargo.lock` (generated at build).

7.5 Bundled third-party executables

Binary	Version	Purpose	License	Source
llama-server (llama.cpp)	release [VERIFY: record build tag b####]	Local AI inference engine	MIT	github.com/ggml-org/llama.cpp
Syft	1.18.1	SBOM generation	Apache-2.0	github.com/anchore/syft
Grype	0.85.0	Dependency CVE scanning	Apache-2.0	github.com/anchore/grype
Grant	0.2.4	License compliance scanning	Apache-2.0	github.com/anchore/grant
Cosign	2.4.1	SBOM signing/verification	Apache-2.0	github.com/sigstore/cosign
WebView2 Runtime (fixed version)	per Microsoft	Web rendering host	Microsoft Software License (redistributable)	Microsoft

7.6 AI model

Attribute	Detail
Name	"astra-code-reviewer" (fine-tune)
Base architecture	Qwen2.5-Coder-3B (Qwen2ForCausalLM): hidden 2048, 36 layers, 16 heads, vocab 151,936, context 32,768
Shipped format	GGUF, q8_0 quantization
File	astra.gguf (bundled), 3.1 GB
SHA-256	e927148c7023a5f5c67ad508cbbb3613a5a9a5cbac4b09a0a999a1b8f59b0586
Function	Generates structured vulnerability findings from code chunks; runs 100% locally
[VERIFY] Licensing	The base Qwen2.5-Coder-3B is published under the Qwen license (the 3B variant historically uses a Qwen Research/Community license, not Apache-2.0). The vendor must confirm commercial-use rights for bank deployment and provide the model license text.

7.7 Embedded datastore

Component	Version	License
SQLite (via Python sqlite3)	bundled with Python 3.13	Public Domain

7.8 Artifact integrity

Artifact	Integrity control
AI model astra.gguf	SHA-256 published above (verify post-transfer).

Artifact	Integrity control
Generated SBOMs	Signed with Cosign (Ed25519) at scan time; verifiable offline.
Windows installer	[VERIFY] Authenticode code-signed by the vendor's certificate (provide cert details + thumbprint).
Final installer SHA-256	[VERIFY] produced on the Windows build host; record and ship alongside the installer.

7.9 Build-time provenance (vendor side — not on the bank laptop)

The installer is assembled on the vendor's build host: dependencies are pinned, the AI model is quantized from the base weights, the scanner binaries + Grype vulnerability-DB snapshot are downloaded, and everything is bundled and signed. **None of these network operations occur on the bank endpoint** — the bank receives only the finished, signed installer.

8. System requirements & installation

Item	Requirement
OS	Windows 10/11 (x64)
CPU	x64; AI scans are CPU-only (no GPU required). More cores = faster scans.
RAM	8 GB minimum, 16 GB recommended (the AI model uses ~3–4 GB while active).
Disk	~4–5 GB installed (model + binaries + runtime).
Privileges	Per-user install (no kernel drivers, no services). Admin rights not required to run. [VERIFY] confirm whether your packaging policy needs a per-machine vs per-user MSI/NSIS.
Runtime prerequisites	Bundled (Python runtime, WebView2 fixed runtime). No separate installs.
Install footprint	App in Program Files (or per-user); data in %LOCALAPPDATA%\com.codesense.desktop\.

9. Licensing & entitlement

Topic	Detail
Term	90 days from first launch (configurable at build).
Mechanism	Offline. A signed "first-seen" timestamp is recorded locally; no server contact.
Expiry behavior	Read-only grace — after expiry, existing findings remain viewable but new scans and edits are disabled.
Tamper resistance	A signed local stamp (file + registry) plus a system-clock-rollback guard. Honest scope: this is deterrence against casual date-changing/copying, not cryptographic DRM against a determined attacker.
Renewal	A new build/license is issued by the vendor (offline); no network dependency.

10. Operations & lifecycle

Topic	Detail
Start / stop	Launched from Start Menu; a system-tray menu offers Open / Pause AI engine (frees RAM) / Quit . Closing the window minimizes to tray; Quit cleanly stops all components.
Logs	Written locally under the data directory; no remote log shipping.
Backup	Back up %LOCALAPPDATA%\com.codesense.desktop\app.sqlite3 to retain findings.
Updates	Delivered as a new signed installer (no auto-update / no network update channel).
Uninstall	Standard Windows uninstall; delete the data directory to purge all local data.

11. Internal security assessment summary

A focused security review of the application changes was performed (injection, authn/authz, crypto/secrets, deserialization, data exposure).

- **Result:** no concretely-exploitable vulnerabilities identified.
- Database access uses the parameterized Django ORM (no raw SQL); subprocess calls to scanner binaries use argument lists (no shell interpretation); no pickle/yaml/eval deserialization; the React UI uses no unsafe HTML sinks.
- **Hardening applied:** request gating uses exact path-segment matching; the offline-license key is derived per-install from the unique SECRET_KEY rather than a shared constant; first-admin creation enforces password strength and is transaction-guarded.
- **Test coverage:** 59 automated backend tests pass.

Vulnerability management note: Gripe CVE data is bundled as a **point-in-time snapshot** taken at build. It does not auto-update offline; refreshed CVE data requires a new build. (This is a deliberate trade-off for the air-gapped model.)

12. Risk register & limitations (full disclosure)

#	Item	Impact	Mitigation / status
1	Model license [VERIFY]	Legal/compliance	Vendor to confirm Qwen2.5-Coder-3B commercial-use rights + supply license text before sign-off.
2	Installer code-signing [VERIFY]	Endpoint trust	Vendor to Authenticode-sign and provide certificate details + installer SHA-256.
3	License tamper model is deterrence, not DRM	Entitlement	Accepted by design; not a data-security risk.
4	Gripe CVE DB is frozen at build	Stale CVE coverage	Refresh via rebuild on a defined cadence (e.g., quarterly).
5	AI model is q8_0 (~3.1 GB)	Larger/slower than q4	Functional; a smaller Q4_K_M build is available if size/perf matters.

#	Item	Impact	Mitigation / status
6	DB encryption relies on endpoint FDE	Data at rest	Confirm BitLocker policy on target laptops.
7	Windows installer build not yet executed/validated in this package	Delivery readiness	Build + smoke-test on a Windows host per the included BUILD.md before distribution.
8	AI findings may contain false positives/negatives	Review quality	Tool augments, does not replace, human review; findings are advisory.

13. Compliance mapping (summary)

Control area	How Code Sense addresses it
Data residency / sovereignty	All processing and storage on-device; no egress.
Third-party data sharing	None.
Network exposure	No listening ports beyond loopback; no outbound traffic.
Access control	Per-install admin bootstrap, RBAC, bcrypt, JWT.
Supply-chain integrity	Pinned dependencies; bundled SBOM tooling; Cosign-signed SBOMs; (vendor) Authenticode-signed installer.
Auditability	Local findings DB + logs; CycloneDX SBOM available on request.
Least privilege	Per-user, no drivers/services, admin not required to run.

14. Support & contacts

Item	Detail
Vendor	Astra (repository: github.com/grv-astra/yacm) [VERIFY contact/SLA]
Support channel	[VERIFY]
Patch/update cadence	[VERIFY] (recommend quarterly rebuild for refreshed CVE data)
Escalation	[VERIFY]

Appendix A — Regenerating a machine-readable SBOM

The product bundles Syft; an authoritative CycloneDX SBOM of any component can be produced offline:

```
syft dir:<path-to-component> -o cyclonedx-json=code-sense-sbom.json
cosign sign-blob code-sense-sbom.json --key cosign.key --bundle code-sense-sbom.bundle.json
```

Appendix B — Exact backend dependency manifest

The Python manifest in §7.2 is the live `pip freeze` of the shipped virtual environment

(Python 3.13.1). The frontend (`package-lock.json/bun.lock`) and Rust (`Cargo.lock`) lockfiles

pin the complete transitive trees and can be provided in full on request.